NanoNet: Ultra-Lightweight Network for MRI Brain Tumor Classification toward Edge AI

Le-Tran Nguyen[1,2], Huu-Thuan Huynh[1,2], Minh-Triet Tran[1,2], and Trung-Nghia Le[1,2]*

[1] University of Science, Ho Chi Minh City, Vietnam
[2] Vietnam National University, Ho Chi Minh City, Vietnam
nltran2525@apcs.fitus.edu.vn, hhthuan@hcmus.edu.vn, {tmtriet,ltnghia}@fit.hcmus.edu.vn

Abstract. Automated brain tumor classification from magnetic resonance imaging (MRI) can support accessible medical image analysis, but conventional deep learning models often exceed the resource budgets of highly constrained edge-AI platforms. We present NanoNet, an ultra-lightweight convolutional neural network designed for resource-constrained MRI classification and motivated by future edge-AI applications. Inspired by MobileNetV2, NanoNet substantially reduces network depth, channel capacity, expansion ratios, and final feature dimensionality, resulting in only five inverted residual blocks and 8,132 parameters. We further investigate dependency-aware structured pruning, knowledge distillation, and INT8 quantization-aware training for additional compression. Experiments on the BRISC 2025 benchmark show that NanoNet achieves 92.8% accuracy with 8,132 parameters and 0.0105 GFLOPs. At the same computational cost, it outperforms 70%-pruned MobileNetV2 by 2.9 percentage points while using 58.6% fewer parameters. Its INT8 TensorFlow Lite model requires only 27.7 KB of storage, demonstrating strong potential for future edge-AI deployment.

Keywords: Brain tumor classification, Lightweight neural networks, Model compression, Edge AI

1 Introduction

Brain tumors are challenging to analyze in medical imaging because their appearance varies across tumor types, anatomical locations, imaging protocols, and patients. Magnetic resonance imaging (MRI) provides rich structural and contrast information and is widely used for brain tumor assessment. Automated MRI classification using deep learning has consequently become an active research direction, with CNN-based methods demonstrating strong performance in distinguishing glioma, meningioma, and pituitary tumors [2,3,17].

Despite their predictive capability, many deep learning models for medical image analysis are designed for computational environments with substantially

* Corresponding author.

more resources than highly constrained embedded platforms [7]. Their parameter count, computational complexity, and memory requirements can make local inference difficult on resource-limited hardware. Edge AI offers an attractive alternative by enabling computation closer to the data source with reduced dependence on cloud infrastructure [22]. However, realizing this potential requires models that maintain useful accuracy under extremely small computational, memory, and storage budgets, as emphasized by platform-aware adaptation and tiny-device neural networks [20,14].

Lightweight architectures such as MobileNet [11] and MobileNetV2 [18] reduce computation through depthwise convolutions and inverted residual blocks, while subsequent designs such as ShuffleNet [21] and MobileNetV3 [10] further optimize lightweight networks for mobile and embedded platforms. However, these architectures may still retain unnecessary capacity under extreme resource constraints. Structured pruning and quantization can provide further reductions, although aggressive post-hoc compression may substantially degrade predictive performance. This motivates an architecture-first strategy: instead of starting from a conventional lightweight network and relying primarily on aggressive compression, we design a compact architecture specifically for an extreme resource regime and subsequently investigate additional compression techniques.

To this end, we present NanoNet, an ultra-lightweight CNN for resource-constrained MRI brain tumor classification. Inspired by MobileNetV2, NanoNet substantially reduces architectural capacity by using only eight initial channels, five inverted residual blocks, expansion ratios $t \in \{1, 2\}$, and a 48-channel final representation. We further investigate a deployment-oriented compression pipeline combining dependency-aware structured pruning, knowledge-distillation-based fine-tuning, and INT8 quantization-aware training. On the BRISC 2025 benchmark, NanoNet achieves 92.8% accuracy with only 8,132 parameters and 0.0105 GFLOPs, providing a competitive accuracy0efficiency operating point under an extremely constrained computational budget. These results indicate strong potential for future edge-AI implementation.

Our contributions are as follows:

- We introduce NanoNet, an ultra-lightweight CNN specifically designed for extremely resource-constrained MRI brain tumor classification. The architecture adopts an architecture-first design that substantially reduces network depth, channel capacity, expansion ratios, and final feature dimensionality relative to MobileNetV2.
- We investigate a deployment-oriented compression pipeline that combines dependency-aware structured pruning, knowledge-distillation-based fine-tuning, and INT8 quantization-aware training. We further characterize the resulting models in terms of computational cost, runtime activation memory, and persistent model storage.
- We systematically evaluate the accuracy0efficiency trade-off between NanoNet and MobileNetV2 0.1× under different computational budgets, demonstrating the advantage of architecture-first reduction in the extreme low-computation regime.

2 Related Work

Deep Learning for Brain Tumor Classification. CNNs have been widely applied to automated brain tumor classification from MRI [2,3,17]. While these methods achieve strong predictive performance, they primarily emphasize classification accuracy and often rely on relatively large CNN architectures, leaving computational and memory efficiency less explored. This motivates studying brain tumor classification under an explicit accuracy–efficiency trade-off, particularly for highly resource-constrained edge-AI platforms.

Lightweight Networks for Edge AI. MobileNet [11] introduced depthwise separable convolutions for efficient mobile vision, while MobileNetV2 [18] further improved efficiency through inverted residuals and linear bottlenecks. Subsequent architectures, including ShuffleNet [21], MobileNetV3 [10], and GhostNet [6], explored efficient channel operations, architecture search, and inexpensive feature generation for resource-constrained devices. Platform-aware approaches such as NetAdapt [20] and MCUNet [14] further emphasize deployment constraints beyond conventional FLOPs and parameter counts. These studies motivate our architecture-first strategy, which targets an extremely small computational regime before applying additional compression.

Structured Compression and Quantization. Structured pruning reduces computational cost by removing complete filters or channels rather than individual weights. Representative approaches include filter pruning [13], network slimming [15], and ThiNet [16], while recent surveys provide a broader analysis of structured pruning methods and their hardware implications [8]. Knowledge distillation [9] provides an additional mechanism for recovering predictive performance in compact models, and subsequent studies have examined practical factors affecting distillation effectiveness [1]. For low-precision deployment, integer-arithmetic inference and quantization-aware training have been extensively studied [12], with hardware-aware quantization further incorporating deployment constraints into the quantization process [19].

3 Proposed Method

3.1 Design Objective

Our goal is to construct a CNN whose resource characteristics make it a promising candidate for future edge-AI applications while maintaining sufficient classification capability. We therefore target three requirements: (1) Low computational cost, to support inference under highly constrained processing budgets; (2) Low model-storage requirements, to support platforms with limited persistent storage; (3) Sufficient classification capacity, to maintain useful performance for four-class MRI classification.

Rather than treating these requirements solely as post-training optimization objectives, we incorporate them directly into the architecture. This architecture-first design is intended to establish an extremely small computational baseline before applying additional compression. This perspective is consistent with prior

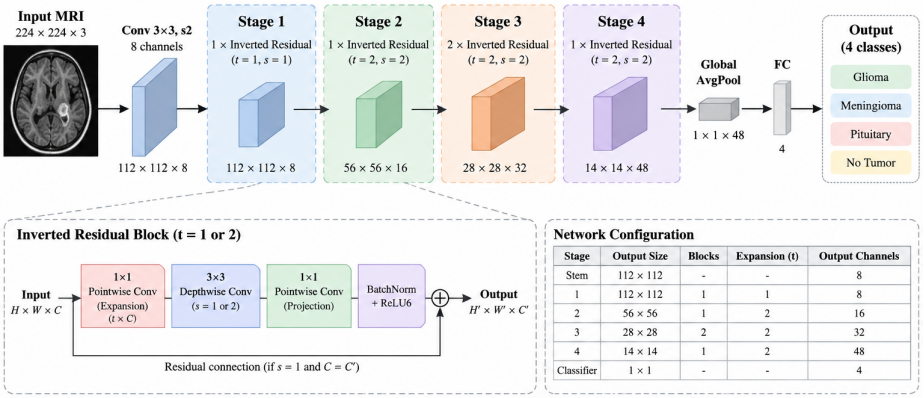


Fig. 1: Overview of the NanoNet architecture. Starting from a $224 \times 224 \times 3$ MRI image, NanoNet employs a compact convolutional stem followed by five inverted residual blocks organized into four stages. The architecture uses reduced channel capacity and expansion ratios $t \in \{1, 2\}$, followed by global average pooling and a four-class classification head.

work on platform-aware network adaptation [20] and tiny-device neural networks [14], which emphasize designing models around stringent deployment constraints rather than relying solely on post-hoc compression.

3.2 NanoNet Architecture

As illustrated in Fig. 1, NanoNet adopts the inverted residual and depthwise convolution design of MobileNetV2 [18], while substantially reducing architectural capacity for extreme resource constraints. This architecture-first design minimizes capacity before applying further compression:

- *Initial Channel Capacity Reduction*: The first convolutional layer produces only 8 channels, compared with 32 channels in the reference configuration.
- *Depth Reduction*: The architecture is reduced from 19 inverted residual blocks across seven stages to five blocks across four stages.
- *Restricted Expansion Ratios*: To prevent internal parameter explosion, the expansion ratio in each Inverted Residual Block is tightly constrained from the standard $t = 6$ to $t \in \{1, 2\}$.
- *Compact Final Representation*: Instead of expanding the final representation to 1,280 channels, NanoNet uses only 48 channels before classification.

Lightweight Inverted Residual Block. NanoNet adopts the inverted residual design of MobileNetV2 as its fundamental building block. Unlike conventional residual blocks that first reduce and then expand the feature dimension, an inverted residual block expands the input channels into a higher-dimensional intermediate representation, applies a depthwise convolution, and subsequently projects the features back to a compact output representation.

Given an input feature map $\mathbf{x} \in \mathbb{R}^{H\times W\times C}$, an inverted residual block with expansion ratio t first applies a 1×1 pointwise convolution to expand the channel dimension from C to tC. A 3×3 depthwise convolution then processes each channel independently, followed by a second 1×1 pointwise convolution that projects the representation to the output dimension C'. Batch normalization and ReLU6 activations are applied to the intermediate transformations. Formally, the block can be expressed as: $\mathbf{z} = \phi_{\text{proj}}\left(\text{PW}_{\text{proj}}\left(\text{DW}_{3\times 3}\left(\phi_{\text{exp}}\left(\text{PW}_{\text{exp}}(\mathbf{x})\right)\right)\right)\right)$, where PW_{exp} and PW_{proj} denote the expansion and projection pointwise convolutions, respectively, $\text{DW}_{3\times 3}$ denotes the depthwise convolution, and ϕ represents the corresponding normalization and activation operations.

A residual connection is employed when the input and output feature maps have identical spatial dimensions and channel numbers. Thus,

$$\mathbf{y} = \begin{cases} \mathbf{x} + \mathbf{z}, & \text{if } s = 1 \text{ and } C = C', \\ \mathbf{z}, & \text{otherwise,} \end{cases} \tag{1}$$

where s denotes the stride of the depthwise convolution.

To make the block suitable for highly constrained edge devices, NanoNet substantially restricts the expansion ratio. While the original MobileNetV2 commonly uses $t = 6$, NanoNet limits the expansion ratio to $t \in \{1, 2\}$. This design reduces the number of intermediate channels and consequently lowers both parameter count and computational cost. Combined with the reduced network depth and channel capacity described in Section 3.2, the resulting inverted residual blocks provide sufficient feature transformation capacity while maintaining a compact computational footprint.

3.3 Deployment-Oriented Compression

Overview. Although the architecture-first design of NanoNet substantially reduces the computational and parameter budgets, further compression can provide additional resource savings. We therefore investigate a deployment-oriented compression pipeline consisting of structured pruning [8,13,15], knowledge distillation (KD) [9], and INT8 quantization-aware training (QAT) [12].

The overall pipeline is illustrated in Fig. 2. The pre-compression network is used as a teacher throughout the compression process. The compact model is progressively optimized while undergoing structural pruning and low-precision adaptation. In this way, the original model provides a stable source of predictive knowledge for recovering performance after each compression operation.

Structured Pruning. NanoNet is further compressed using iterative structured pruning. Rather than removing individual weights, which may introduce irregular sparsity with limited practical acceleration, we remove entire coupled channels to obtain a structurally compact network.

We employ the DepGraph framework [4] to identify dependencies between coupled parameters. This is particularly important for inverted residual blocks, where the expansion, depthwise, and projection layers are structurally coupled.

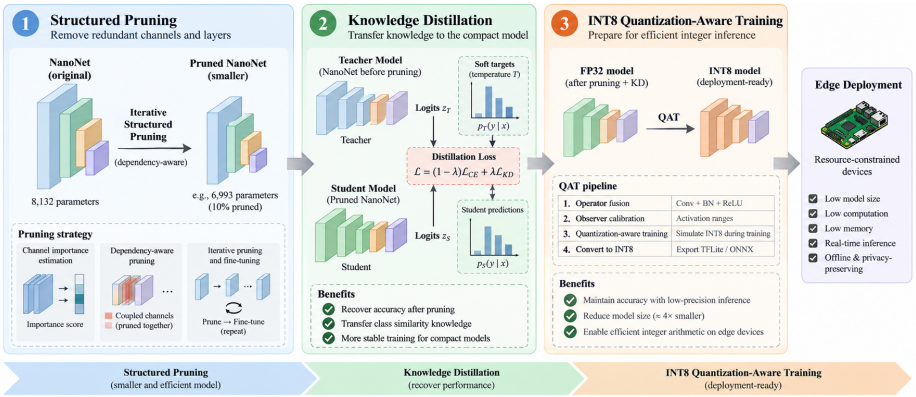


Fig. 2: Overview of compression pipeline. The original NanoNet is progressively compressed through dependency-aware structured pruning, knowledge-distillation-based fine-tuning, and INT8 quantization-aware training (QAT), producing a compact model with deployment-oriented resource characteristics.

Directly pruning a single layer without accounting for these dependencies can lead to incompatible tensor dimensions or invalid residual connections.

Given a target pruning ratio r, channel importance is first estimated and the corresponding coupled parameters are grouped according to the dependency graph. The reported pruning ratio denotes the target channel-removal ratio supplied to the dependency-aware pruning procedure rather than the resulting percentage reduction in total parameters. Because coupled layers are pruned jointly, the resulting parameter and computational reductions can differ from the nominal pruning ratio.

The selected groups are then removed simultaneously. The resulting network is subsequently fine-tuned before the next pruning step. This process is repeated iteratively to progressively reduce the model size while preserving the structural integrity of NanoNet.

Formally, let $\mathcal{M}_0$ denote the original NanoNet and $\mathcal{M}_k$ denote the model after the k-th pruning step. The iterative procedure can be summarized as

$$\mathcal{M}_{k+1} = \mathcal{P}(\mathcal{M}_k, r_k), \tag{2}$$

where $\mathcal{P}(\cdot)$ represents dependency-aware structured pruning and r_k is the pruning ratio at step k. After each pruning operation, the resulting model is fine-tuned using knowledge distillation from the original pre-compression model.

Knowledge Distillation. Aggressive pruning can substantially reduce the representational capacity of a compact network and consequently degrade classification accuracy. To compensate for this degradation, we employ knowledge distillation, in which the original pre-compression NanoNet acts as the teacher and the pruned model acts as the student.

Let $\mathbf{z}_T$ and $\mathbf{z}_S$ denote the logits produced by the teacher and student networks, respectively. The softened class distributions are computed as

$$p_T(y \mid x) = \text{softmax}\left(\frac{\mathbf{z}_T}{T}\right), \qquad p_S(y \mid x) = \text{softmax}\left(\frac{\mathbf{z}_S}{T}\right), \tag{3}$$

where T is the temperature parameter.

The student is optimized using a combination of the standard classification loss and the distillation loss:

$$\mathcal{L} = (1-\lambda)\mathcal{L}_{CE} + \lambda\mathcal{L}_{KD}, \tag{4}$$

where $\mathcal{L}_{CE}$ is the cross-entropy loss, $\mathcal{L}_{KD}$ measures the discrepancy between the teacher and student predictions, and λ controls the contribution of knowledge distillation.

In our compression pipeline, KD is applied after each pruning step during fine-tuning. Consequently, the teacher provides a stable supervisory signal while the student progressively adapts to its reduced architecture. This compression-aware strategy helps recover classification performance without increasing the inference-time complexity of the final model.

Quantization-Aware Training. After structural pruning and knowledge-distillation-based fine-tuning, the resulting FP32 model is converted into an INT8 model for efficient edge inference. We employ quantization-aware training (QAT) [12] to expose the network to quantization effects during training and reduce the accuracy degradation caused by low-precision arithmetic.

Before QAT, consecutive convolution, batch normalization, and activation operations are fused where applicable. Residual connections are also adapted to ensure compatibility with integer arithmetic. Activation observers are subsequently calibrated using a small subset of the training data to estimate appropriate activation ranges.

During QAT, fake-quantization operations simulate INT8 rounding and saturation while the underlying parameters remain trainable. The model therefore learns to compensate for the numerical errors introduced by low-precision representation. We use a learning rate of 1×10^{-5} during the QAT stage.

After QAT, the trained network is converted to an 8-bit integer representation suitable for deployment-oriented evaluation. The resulting model provides a low-storage integer representation that can facilitate future implementation on resource-constrained edge-AI hardware.

4 Experiments

4.1 Experimental Settings

Dataset. We evaluate NanoNet on the BRISC 2025 dataset [5], which contains 6,000 T1-weighted brain MRI images from four classes: Glioma, Meningioma, Pituitary Tumor, and No Tumor. Following the provided benchmark split, 5,000 images are used for training and 1,000 for testing.

Evaluation Metrics. We evaluate models in terms of classification accuracy and resource efficiency. Accuracy is measured on the test set for both FP32 models and their corresponding INT8 versions. Resource efficiency is characterized by the number of trainable parameters and computational complexity in GFLOPs at 224×224 input resolution. For the INT8 models, we additionally report peak TensorFlow Lite Tensor Arena memory during single-batch inference and persistent model storage measured by the size of the converted `.tflite` model. These measurements characterize deployment-oriented resource requirements rather than performance on a physical edge device.

Baseline. We use MobileNetV2 0.1× [18] as the primary baseline because it follows the same inverted residual design philosophy as NanoNet while providing a substantially larger lightweight architecture. To evaluate the benefit of architecture-first reduction under extreme resource constraints, we compare NanoNet with MobileNetV2 under multiple structured-pruning ratios rather than considering only the uncompressed baseline.

Implementation Details. All models are implemented in PyTorch and trained and evaluated using dual NVIDIA Tesla T4 GPUs with an Intel x86-64 CPU. All experiments use an input resolution of 224×224. Parameter counts and computational complexity are measured using the torch-pruning profiling utility. For the compression experiments, models are first evaluated after structured pruning and knowledge-distillation-based fine-tuning, followed by INT8 conversion through QAT. QAT uses a learning rate of 1×10^{-5} before the final INT8 conversion.

4.2 Results

Classification Performance and Computational Efficiency. Table 1a compares MobileNetV2 0.1× and NanoNet under different structured-pruning configurations. The results reveal a clear trade-off between computational reduction and classification performance.

The uncompressed MobileNetV2 0.1× achieves the highest accuracy of 97.8%, but requires 91,204 parameters and 0.0630 GFLOPs. In contrast, NanoNet is substantially smaller by design, containing only 8,132 parameters and requiring 0.0105 GFLOPs while achieving 92.8% accuracy. Thus, NanoNet reduces the parameter count by approximately 91.1% and computational cost by approximately 83.3% relative to the MobileNetV2 baseline.

More importantly, the comparison becomes particularly relevant in the extreme low-computation regime. At the reported 0.0105-GFLOP operating point, NanoNet achieves 92.8% accuracy, whereas the 70%-pruned MobileNetV2 achieves 89.9%. NanoNet therefore improves accuracy by 2.9 percentage points while using only 8,132 parameters compared with 19,631 parameters for the pruned MobileNetV2, corresponding to a 58.6% reduction in parameter count.

This result does not indicate that NanoNet is universally more accurate than MobileNetV2. Rather, it shows that architecture-first reduction can provide a more favorable accuracy-efficiency operating point when the computational budget is extremely constrained.

Table 1: Classification performance and resource characteristics of MobileNetV2 0.1× and NanoNet under structured pruning and quantization-aware training.

(a) Acc. and computational efficiency.

Model	#Params	GFLOPs	Accuracy
MobileNetV2 0.1×			
Baseline	91,204	0.0630	97.8%
30% pruned	59,931	0.0337	97.4%
50% pruned	41,840	0.0234	96.5%
60% pruned	29,699	0.0181	94.2%
70% pruned	19,631	0.0105	89.9%
75% pruned	15,716	0.0105	85.4%
90% pruned	4,401	0.0029	71.9%
NanoNet			
Baseline	8,132	0.0105	92.8%
10% pruned	6,993	0.0064	89.0%
25% pruned	5,396	0.0029	69.5%

(b) Acc. before and after QAT.

Model	FP32	INT8	Δ
MobileNetV2 0.1×			
Baseline	97.8%	97.8%	0.0%
30% Pruned	97.4%	97.8%	+0.4%
50% Pruned	96.5%	95.5%	-1.0%
60% Pruned	94.2%	93.8%	-0.4%
70% Pruned	89.9%	88.2%	-1.7%
75% Pruned	85.4%	83.9%	-1.5%
90% Pruned	71.9%	72.0%	+0.1%
NanoNet			
Baseline	92.8%	90.1%	-2.7%
10% Pruned	89.0%	86.3%	-2.7%
25% Pruned	69.5%	70.2%	+0.7%

Accuracy-Efficiency Trade-off. Fig. 3 illustrates the relationship between classification accuracy and computational complexity. MobileNetV2 maintains high accuracy at relatively larger computational budgets, but its performance decreases substantially under aggressive pruning.

NanoNet operates in a substantially smaller computational regime. Its baseline configuration achieves 92.8% accuracy at only 0.0105 GFLOPs, whereas further pruning to 0.0064 and 0.0029 GFLOPs results in 89.0% and 69.5% accuracy, respectively.

These results indicate that architecture-first reduction can provide a more favorable operating point than applying aggressive pruning to a relatively larger lightweight architecture. In particular, NanoNet achieves substantially higher accuracy than heavily pruned MobileNetV2 at comparable computational complexity.

INT8 Quantization. Table 1b compares FP32 and INT8 accuracy after QAT. Quantization affects the two architectures differently. MobileNetV2 exhibits relatively small changes across the evaluated configurations, whereas NanoNet is more sensitive to INT8 conversion because its representation is already highly compact.

For NanoNet, the baseline accuracy decreases from 92.8% in FP32 to 90.1% after INT8 conversion, corresponding to a 2.7-percentage-point reduction. The 10%-pruned configuration exhibits the same reduction, from 89.0% to 86.3%, while the 25%-pruned configuration increases slightly from 69.5% to 70.2%.

These results show that low-precision conversion can introduce non-negligible accuracy changes for extremely compact networks. Nevertheless, the resulting INT8 models provide substantially smaller integer representations, supporting

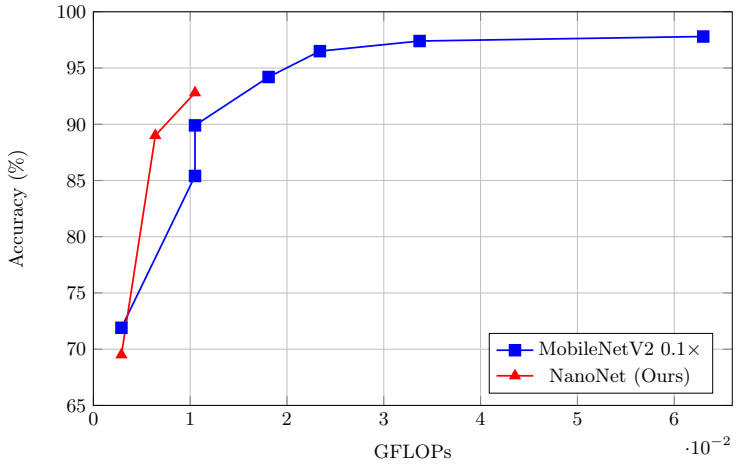


Fig. 3: Accuracy-computational-cost trade-off between MobileNetV2 0.1× and NanoNet under different structured-pruning configurations. NanoNet operates in a substantially lower computational regime and provides a competitive accuracy operating point under extreme resource constraints.

Table 2: Runtime activation memory and persistent model-storage footprint of selected INT8 models. Peak Tensor Arena represents the maximum TensorFlow Lite runtime memory required during single-batch inference, while Model Storage represents the size of the converted `.tflite` model.

Model	Peak Tensor Arena (KB)	Model Storage (KB)
MobileNetV2 0.1×		
70% Pruned	333.4	81.4
60% Pruned	333.4	103.0
NanoNet		
Baseline	394.6	27.7
10% Pruned	382.4	27.6

the potential of NanoNet for future resource-constrained edge-AI implementations.

Memory and Model Storage. Table 2 provides a deployment-oriented analysis of the selected INT8 models. The baseline NanoNet requires only 27.7 KB of model storage after INT8 conversion, while the 10%-pruned NanoNet requires 27.6 KB. In comparison, the 70%- and 60%-pruned MobileNetV2 models require 81.4 KB and 103.0 KB, respectively.

The runtime activation memory exhibits a different behavior. NanoNet requires 394.6 KB of peak Tensor Arena memory in the baseline configuration and 382.4 KB after 10% pruning, compared with 333.4 KB for the selected MobileNetV2 configurations. This result highlights an important distinction be-

tween persistent model storage and runtime activation memory: reducing parameter count and model-file size does not necessarily minimize peak activation memory.

Although these measurements are not obtained on a physical edge device, the extremely small computational footprint and model-storage requirements demonstrate that NanoNet has strong potential for future implementation on highly constrained edge-AI platforms.

4.3 Analysis of Resource-Accuracy Trade-offs

Pruning-Ratio Analysis. MobileNetV2 maintains relatively high accuracy under moderate pruning but degrades rapidly under aggressive pruning. NanoNet is more sensitive to additional pruning: 10% pruning reduces its accuracy from 92.8% to 89.0%, while 25% pruning reduces it to 69.5%. This behavior indicates that the architecture-first design leaves limited redundancy for further structural removal.

Compression Pipeline Analysis. The compression stages address complementary resource constraints. Structured pruning reduces channel capacity and consequently lowers parameter count and computational cost, while knowledge distillation (KD) provides supervisory information during post-pruning fine-tuning. INT8 quantization-aware training (QAT) subsequently adapts the compact FP32 models to low-precision inference and enables a smaller integer representation.

For NanoNet, additional pruning reduces the computational cost from 0.0105 GFLOPs to 0.0064 and 0.0029 GFLOPs at 10% and 25% nominal pruning ratios, respectively. However, accuracy decreases from 92.8% to 89.0% and 69.5%, indicating that the compact baseline contains limited redundancy and is sensitive to further structural reduction. After QAT, the baseline and 10%-pruned NanoNet models experience a 2.7-percentage-point accuracy reduction, whereas the 25%-pruned model changes slightly from 69.5% to 70.2%.

These results highlight the complementary roles of the compression stages while also revealing the limitations of aggressive compression for an already compact architecture. Since the current experiments do not include a separate no-KD ablation, the individual contribution of KD cannot be isolated. Nevertheless, the complete pipeline produces highly compact models, with the baseline NanoNet requiring only 27.7 KB of INT8 model storage.

Resource-Budget Analysis. The evaluated models span computational budgets from 0.0630 GFLOPs for the original MobileNetV2 0.1× to 0.0029 GFLOPs for the most aggressively compressed configurations. The comparison at 0.0105 GFLOPs is particularly informative: NanoNet achieves 92.8% accuracy with 8,132 parameters, whereas 70%-pruned MobileNetV2 achieves 89.9% accuracy with 19,631 parameters. Thus, NanoNet provides a 2.9-percentage-point accuracy improvement while using 58.6% fewer parameters at the same reported computational cost.

The advantage of NanoNet is specific to this extreme low-resource regime rather than universal. Uncompressed MobileNetV2 0.1× achieves 97.8% accuracy at 0.0630 GFLOPs, but its accuracy declines substantially under aggressive pruning. In contrast, NanoNet starts from a much smaller architecture and retains 92.8% accuracy at 0.0105 GFLOPs. Further pruning NanoNet to 0.0064 and 0.0029 GFLOPs causes substantial accuracy degradation, confirming that its baseline configuration represents a favorable operating point among the evaluated models.

From a deployment perspective, the baseline NanoNet also requires only 27.7 KB of INT8 model storage. However, these results characterize deployment-oriented resource requirements rather than physical edge-device performance; hardware-level measurements of latency, throughput, power, and energy remain necessary for practical deployment.

4.4 Limitations

Despite its compact design, NanoNet has four main limitations. First, evaluation on a single benchmark does not establish cross-dataset generalization. Second, the experiments use 2D MRI images rather than volumetric 3D data. Third, the small architectural capacity leaves limited redundancy, making further pruning highly sensitive. Finally, we do not evaluate NanoNet on physical ARM, mobile, or embedded hardware; thus, latency, throughput, power, and energy measurements remain future work.

5 Conclusion

We presented NanoNet, an ultra-lightweight CNN for resource-constrained MRI brain tumor classification. Its architecture-first design reduces network depth, channel capacity, expansion ratios, and feature dimensionality, achieving 92.8% accuracy with only 8,132 parameters and 0.0105 GFLOPs. At the same computational cost, NanoNet outperforms 70%-pruned MobileNetV2 by 2.9 percentage points while using 58.6% fewer parameters. The results show that architecture-first design provides a favorable accuracy–efficiency trade-off under extreme resource constraints. Although physical edge-device deployment remains outside the current study, the 27.7 KB INT8 model demonstrates strong potential for such platforms. Future work will evaluate NanoNet on representative hardware and extend the approach to volumetric MRI analysis.

Acknowledgments. This research is funded by Vietnam National University - Ho Chi Minh City (VNU-HCM) under Grant Number A2025-18-04.

References

1. Beyer, L., Zhai, X., Royer, A., Markeeva, L., Anil, R., Kolesnikov, A.: Knowledge distillation: A good teacher is patient and consistent. In: 2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). pp. 10915–10924. IEEE (2022)

2. Cheng, J., Huang, W., Cao, S., Yang, R., Yang, W., Yun, Z., Wang, Z., Feng, Q.: Enhanced performance of brain tumor classification via tumor region augmentation and partition. PloS one **10**(10), e0140381 (2015)
3. Deepak, S., Ameer, P.: Brain tumor classification using deep cnn features via transfer learning. Computers in biology and medicine **111**, 103345 (2019)
4. Fang, G., Ma, X., Song, M., Mi, M.B., Wang, X.: Depgraph: Towards any structural pruning. In: 2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). pp. 16091–16101. IEEE (2023)
5. Fateh, A., Rezvani, Y., Moayedi, S., Rezvani, S., Fateh, F., Fateh, M., Abolghasemi, V.: Brisc: Annotated dataset for brain tumor segmentation and classification. Scientific Data **13**(1) (Feb 2026). https://doi.org/10.1038/s41597-026-06753-y, http://dx.doi.org/10.1038/s41597-026-06753-y
6. Han, K., Wang, Y., Tian, Q., Guo, J., Xu, C., Xu, C.: Ghostnet: More features from cheap operations. In: 2020 IEEE/CVF conference on computer vision and pattern recognition (CVPR). pp. 1577–1586. ieee (2020)
7. Han, S., Mao, H., Dally, W.J.: Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. arXiv preprint arXiv:1510.00149 (2015)
8. He, Y., Xiao, L.: Structured pruning for deep convolutional neural networks: A survey. IEEE transactions on pattern analysis and machine intelligence **46**(5), 2900–2919 (2023)
9. Hinton, G., Vinyals, O., Dean, J.: Distilling the knowledge in a neural network. arXiv preprint arXiv:1503.02531 (2015)
10. Howard, A., Sandler, M., Chu, G., Chen, L.C., Chen, B., Tan, M., Wang, W., Zhu, Y., Pang, R., Vasudevan, V., et al.: Searching for mobilenetv3. In: Proceedings of the IEEE/CVF international conference on computer vision. pp. 1314–1324 (2019)
11. Howard, A.G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., Adam, H.: Mobilenets: Efficient convolutional neural networks for mobile vision applications. arXiv preprint arXiv:1704.04861 (2017)
12. Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., Kalenichenko, D.: Quantization and training of neural networks for efficient integer-arithmetic-only inference. In: 2018 IEEE/CVF conference on computer vision and pattern recognition. pp. 2704–2713. IEEE (2018)
13. Li, H., Kadav, A., Durdanovic, I., Samet, H., Graf, H.P.: Pruning filters for efficient convnets. arXiv preprint arXiv:1608.08710 (2016)
14. Lin, J., Chen, W.M., Lin, Y., Gan, C., Han, S., et al.: Mcunet: Tiny deep learning on iot devices. Advances in neural information processing systems **33**, 11711–11722 (2020)
15. Liu, Z., Li, J., Shen, Z., Huang, G., Yan, S., Zhang, C.: Learning efficient convolutional networks through network slimming. In: 2017 IEEE international conference on computer vision (ICCV). pp. 2755–2763. Ieee (2017)
16. Luo, J.H., Wu, J., Lin, W.: Thinet: A filter level pruning method for deep neural network compression. In: 2017 IEEE international conference on computer vision (ICCV). pp. 5068–5076. IEEE (2017)
17. Reza, A.W., Hossain, M.S., Wardiful, M.A., Farzana, M., Ahmad, S., Alam, F., Nandi, R.N., Siddique, N.: A cnn-based strategy to classify mri-based brain tumors using deep convolutional network. Applied Sciences **13**(1), 312 (2022)
18. Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., Chen, L.C.: Mobilenetv2: Inverted residuals and linear bottlenecks. In: 2018 IEEE/CVF conference on computer vision and pattern recognition. pp. 4510–4520. Ieee (2018)

19. Wang, K., Liu, Z., Lin, Y., Lin, J., Han, S.: Haq: Hardware-aware automated quantization with mixed precision. In: 2019 IEEE/CVF conference on computer vision and pattern recognition (CVPR). pp. 8604–8612. IEEE (2019)
20. Yang, T.J., Howard, A., Chen, B., Zhang, X., Go, A., Sandler, M., Sze, V., Adam, H.: Netadapt: Platform-aware neural network adaptation for mobile applications. In: European conference on computer vision. pp. 289–304. Springer (2018)
21. Zhang, X., Zhou, X., Lin, M., Sun, J.: Shufflenet: An extremely efficient convolutional neural network for mobile devices. In: 2018 IEEE/CVF conference on computer vision and pattern recognition. pp. 6848–6856. IEEE (2018)
22. Zhou, Z., Chen, X., Li, E., Zeng, L., Luo, K., Zhang, J.: Edge intelligence: Paving the last mile of artificial intelligence with edge computing. Proceedings of the IEEE **107**(8), 1738–1762 (2019)